

Tarea 4

Coeficientes Indeterminados vs. Método de Euler (Directo)
para la Solución de EDOs Lineales No Homogéneas de 2.^o Orden
con Coeficientes Constantes

Harold Styven Lagares De Voz

hlagares@utb.edu.co

Docente: Enrique Pereira ebatista@utb.edu.co

11 de abril de 2026

Índice

1. Problema 1 — Método de Coeficientes Indeterminados	2
1.1. Solución General	2
1.1.1. Paso 1: Solución Homogénea y_h	2
1.1.2. Paso 2: Solución Particular y_p por Coeficientes Indeterminados . . .	2
1.1.3. Paso 3: Solución General	3
1.2. Solución del PVI — $y(0) = 2$, $y'(0) = 5$	3
2. Problema 2 — Esquema de Diferencias Finitas Centrales	4
2.1. Parte (a)(i): Derivación del Esquema Iterativo y Valor Inicial	4
2.1.1. Esquema de Diferencias Finitas Centrales	4
2.1.2. Valor Inicial y_1 mediante Diferencias Centrales con Punto Fantasma	5
2.1.3. Resultados Numéricos para $h = 0.5$	6
2.2. Parte (a)(ii): Análisis del Error de Truncamiento	6
2.2.1. Error de Truncamiento Local (ETL)	6
2.2.2. Error de Truncamiento Global (ETG)	7
2.3. Parte (a)(iii): Implementación en Python y Resultados	7
3. Problema 3 — Movimiento de un Proyectoil	12
3.1. Parte (a): Sistema de Ecuaciones Diferenciales	12
3.2. Parte (b): Función Python <code>my_projectile_motion_solver</code>	12

1. Problema 1 — Método de Coeficientes Indeterminados

Considere la EDO lineal no homogénea de segundo orden con coeficientes constantes:

$$y'' + 4y' + 4y = (3 + x)e^{-2x} \quad (1)$$

1.1. Solución General

1.1.1. Paso 1: Solución Homogénea y_h

Resolvemos la ecuación homogénea asociada:

$$y'' + 4y' + 4y = 0$$

Proponiendo $y = e^{rx}$, se obtiene la **ecuación característica**:

$$r^2 + 4r + 4 = 0 \implies (r+2)^2 = 0 \implies r_1 = r_2 = -2 \quad (\text{raíz doble, multiplicidad } s = 2)$$

Para una raíz repetida $r = -2$, la solución homogénea general es:

$$\boxed{y_h = (c_1 + c_2 x)e^{-2x}} \quad (2)$$

1.1.2. Paso 2: Solución Particular y_p por Coeficientes Indeterminados

El lado derecho tiene la forma $g(x) = P_n(x)e^{\alpha x}$, donde $P_n(x) = 3 + x$ es un polinomio de grado $n = 1$ y $\alpha = -2$.

Forma de prueba (sin corrección por raíz):

$$y_p^{(\text{prueba})} = (A_0 + A_1 x)e^{-2x}$$

Dado que $\alpha = -2$ es raíz de la ecuación característica con multiplicidad $s = 2$, se debe multiplicar la forma de prueba por $x^s = x^2$:

$$y_p = x^2(A + Bx)e^{-2x} = (Ax^2 + Bx^3)e^{-2x} \quad (3)$$

Cálculo de y'_p :

$$\begin{aligned} y'_p &= \frac{d}{dx}[(Ax^2 + Bx^3)e^{-2x}] \\ &= e^{-2x}(2Ax + 3Bx^2) + e^{-2x}(-2)(Ax^2 + Bx^3) \\ &= e^{-2x}[-2Bx^3 + (3B - 2A)x^2 + 2Ax] \end{aligned} \quad (4)$$

Cálculo de y''_p :

$$\begin{aligned} y''_p &= \frac{d}{dx}\left\{e^{-2x}[-2Bx^3 + (3B - 2A)x^2 + 2Ax]\right\} \\ &= e^{-2x}[-6Bx^2 + 2(3B - 2A)x + 2A] + e^{-2x}(-2)[-2Bx^3 + (3B - 2A)x^2 + 2Ax] \\ &= e^{-2x}[-6Bx^2 + (6B - 4A)x + 2A + 4Bx^3 - 2(3B - 2A)x^2 - 4Ax] \\ &= e^{-2x}[4Bx^3 + (4A - 12B)x^2 + (6B - 8A)x + 2A] \end{aligned} \quad (5)$$

Sustitución en la Ec. (1):

Formamos $y_p'' + 4y_p' + 4y_p$ y factorizamos e^{-2x} :

$$y_p'' + 4y_p' + 4y_p = e^{-2x} \left\{ \underbrace{[4Bx^3 + (4A - 12B)x^2 + (6B - 8A)x + 2A]}_{y_p''} + 4 \underbrace{[-2Bx^3 + (3B - 2A)x^2 + 2Ax]}_{y_p'} + 4 \underbrace{[Ax^2 + Bx^3]}_{y_p} \right\}$$

Agrupando por potencias de x :

Potencia	Combinación de coeficientes
x^3	$4B - 8B + 4B = 0$
x^2	$(4A - 12B) + 4(3B - 2A) + 4A = 4A - 12B + 12B - 8A + 4A = 0$
x^1	$(6B - 8A) + 4(2A) = 6B$
x^0	$2A$

Por lo tanto:

$$y_p'' + 4y_p' + 4y_p = e^{-2x}(2A + 6Bx) \quad (6)$$

Igualando con el lado derecho $(3 + x)e^{-2x}$:

$$2A = 3 \implies A = \frac{3}{2} \quad (7)$$

$$6B = 1 \implies B = \frac{1}{6} \quad (8)$$

La solución particular es:

$$y_p = e^{-2x} \left(\frac{3}{2}x^2 + \frac{1}{6}x^3 \right) \quad (9)$$

1.1.3. Paso 3: Solución General

$$y(x) = y_h + y_p = e^{-2x} \left(c_1 + c_2 x + \frac{3}{2}x^2 + \frac{1}{6}x^3 \right) \quad (10)$$

1.2. Solución del PVI — $y(0) = 2$, $y'(0) = 5$

Aplicando la condición inicial $y(0) = 2$:

$$y(0) = e^0(c_1 + 0 + 0 + 0) = c_1 = 2$$

Cálculo de $y'(x)$:

$$\begin{aligned} y'(x) &= \frac{d}{dx} \left[e^{-2x} \left(c_1 + c_2 x + \frac{3}{2} x^2 + \frac{1}{6} x^3 \right) \right] \\ &= -2 e^{-2x} \left(c_1 + c_2 x + \frac{3}{2} x^2 + \frac{1}{6} x^3 \right) + e^{-2x} \left(c_2 + 3x + \frac{1}{2} x^2 \right) \end{aligned} \quad (11)$$

Aplicando la condición inicial $y'(0) = 5$:

$$y'(0) = e^0 (-2c_1 + c_2) = -2(2) + c_2 = 5 \implies c_2 = 9$$

Solución del PVI:

$$y(x) = e^{-2x} \left(2 + 9x + \frac{3}{2} x^2 + \frac{1}{6} x^3 \right) \quad (12)$$

2. Problema 2 — Esquema de Diferencias Finitas Centrales

Consideramos el PVI (1)–(12) en $[0, 5]$, discretizado con paso constante h , definiendo la malla $x_i = i h$ para $i = 0, 1, \dots, N$, $N = 5/h$. Para $h = 0.5$ se tiene $N = 10$.

2.1. Parte (a)(i): Derivación del Esquema Iterativo y Valor Inicial

2.1.1. Esquema de Diferencias Finitas Centrales

Aproximamos ambas derivadas de la EDO usando **diferencias finitas centrales de segundo orden**:

$$y'(x_i) \approx \frac{y_{i+1} - y_{i-1}}{2h} + \mathcal{O}(h^2) \quad (13)$$

$$y''(x_i) \approx \frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + \mathcal{O}(h^2) \quad (14)$$

Sustituyendo (13)–(14) en la EDO en el nodo x_i :

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} + 4 \cdot \frac{y_{i+1} - y_{i-1}}{2h} + 4y_i = f_i, \quad f_i = (3 + x_i) e^{-2x_i}$$

Multiplicando por h^2 :

$$(y_{i+1} - 2y_i + y_{i-1}) + 2h(y_{i+1} - y_{i-1}) + 4h^2 y_i = h^2 f_i$$

Reagrupando por y_{i+1} , y_i , y_{i-1} :

$$\underbrace{(1 + 2h)}_{\alpha} y_{i+1} + \underbrace{(-2 + 4h^2)}_{\beta} y_i + \underbrace{(1 - 2h)}_{\gamma} y_{i-1} = h^2 f_i$$

Despejando y_{i+1} :

$$y_{i+1} = \frac{h^2 f_i + (2 - 4h^2) y_i - (1 - 2h) y_{i-1}}{1 + 2h}, \quad i = 1, 2, \dots, N - 1 \quad (15)$$

Coefficientes para $h = 0.5$:

$$\alpha = 1 + 2(0.5) = 2, \quad \beta = -2 + 4(0.25) = -1 \text{ (luego } 2 - 4h^2 = 1), \quad \gamma = 1 - 2(0.5) = 0$$

Para este paso particular, $\gamma = 0$ y el esquema se reduce a:

$$y_{i+1} = \frac{0.25 f_i + y_i}{2}, \quad h = 0.5 \quad (16)$$

Observación: La anulación del término y_{i-1} para $h = 0.5$ es una consecuencia del valor específico de h ; para h más pequeño ambos y_i e y_{i-1} aparecen.

2.1.2. Valor Inicial y_1 mediante Diferencias Centrales con Punto Fantasma

El esquema iterativo (15) requiere tanto y_0 como y_1 . Si bien $y_0 = y(0) = 2$ está dado, $y_1 \approx y(x_1)$ es desconocido. Usamos el **método del punto fantasma**: introducimos un nodo ficticio $x_{-1} = -h$ y aplicamos diferencias centrales en $x_0 = 0$.

Paso 1 — Calcular $y''(x_0)$ desde la EDO:

$$y''(x_0) = f(x_0) - 4y'(x_0) - 4y(x_0) = \underbrace{(3+0)e^0}_3 - 4 \underbrace{(5)}_{y'(0)} - 4 \underbrace{(2)}_{y(0)} = 3 - 20 - 8 = -25$$

Paso 2 — Diferencias centrales en x_0 con el punto fantasma y_{-1} :

$$\frac{y_1 - y_{-1}}{2h} = y'(x_0) = 5 \quad \implies \quad y_1 - y_{-1} = 10h \quad (\text{I})$$

$$\frac{y_1 - 2y_0 + y_{-1}}{h^2} = y''(x_0) = -25 \quad \implies \quad y_1 + y_{-1} = -25h^2 + 2(2) = 4 - 25h^2 \quad (\text{II})$$

Paso 3 — Eliminar y_{-1} sumando (I) y (II):

$$2y_1 = 10h + 4 - 25h^2$$

$$\boxed{y_1 = \frac{4 + 10h - 25h^2}{2}} \quad (17)$$

Para $h = 0.5$:

$$y_1 = \frac{4 + 10(0.5) - 25(0.25)}{2} = \frac{4 + 5 - 6.25}{2} = \frac{2.75}{2} = \mathbf{1.375}$$

Verificación (solución exacta en $x_1 = 0.5$):

$$y(0.5) = e^{-1}(2 + 4.5 + 0.375 + 0.02083) = e^{-1}(6.89583) \approx 2.5370$$

El error de arranque es $|y_1 - y(x_1)| \approx 1.162$; es grande porque $h = 0.5$ no es suficientemente pequeño. Disminuirá a medida que $h \rightarrow 0$.

2.1.3. Resultados Numéricos para $h = 0.5$

Aplicando (16) iterativamente desde $y_0 = 2$, $y_1 = 1.375$:

Cuadro 1: Solución por DFC vs. solución exacta para $h = 0.5$ en $[0, 5]$.

i	x_i	$y_i^{(\text{num})}$	$y(x_i)^{(\text{exacta})}$	$ e_i = y_i^{(\text{num})} - y(x_i) $
0	0.0	2.000000	2.000000	0.000000
1	0.5	1.375000	2.537045	1.162045
2	1.0	0.848447	1.713571	0.865124
3	1.5	0.491891	0.967688	0.475797
4	2.0	0.273951	0.500477	0.226526
5	2.5	0.148423	0.245789	0.097366
6	3.0	0.078844	0.116485	0.037641
7	3.5	0.041281	0.053828	0.012547
8	4.0	0.021381	0.024363	0.002982
9	4.5	0.010984	0.010869	0.000115
10	5.0	0.005608	0.004782	0.000826

2.2. Parte (a)(ii): Análisis del Error de Truncamiento

2.2.1. Error de Truncamiento Local (ETL)

El *error de truncamiento local* τ_i es el residuo que se obtiene al sustituir la solución exacta $y(x)$ en la fórmula de diferencias finitas. A partir de las expansiones de Taylor:

$$y(x_i + h) = y_i + hy'_i + \frac{h^2}{2}y''_i + \frac{h^3}{6}y'''_i + \frac{h^4}{24}y^{(4)}_i + \mathcal{O}(h^5)$$

$$y(x_i - h) = y_i - hy'_i + \frac{h^2}{2}y''_i - \frac{h^3}{6}y'''_i + \frac{h^4}{24}y^{(4)}_i + \mathcal{O}(h^5)$$

Sumando y restando estas expansiones:

$$\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} = y''_i + \frac{h^2}{12}y^{(4)}_i + \mathcal{O}(h^4) \quad (18)$$

$$\frac{y_{i+1} - y_{i-1}}{2h} = y'_i + \frac{h^2}{6}y'''_i + \mathcal{O}(h^4) \quad (19)$$

Sustituyendo la solución *exacta* en la estencilla de diferencias finitas y usando la EDO:

$$\begin{aligned} \tau_i &= \left[\frac{y_{i+1} - 2y_i + y_{i-1}}{h^2} - y''_i \right] + 4 \left[\frac{y_{i+1} - y_{i-1}}{2h} - y'_i \right] \\ &= \frac{h^2}{12}y^{(4)}_i + \frac{4h^2}{6}y^{(3)}_i + \mathcal{O}(h^4) \\ &= h^2 \left[\frac{y^{(4)}(x_i)}{12} + \frac{2y^{(3)}(x_i)}{3} \right] + \mathcal{O}(h^4) \end{aligned} \quad (20)$$

$$\boxed{E_l = \tau_i = \mathcal{O}(h^2) \quad (\text{error de truncamiento local de segundo orden})} \quad (21)$$

2.2.2. Error de Truncamiento Global (ETG)

El *error de truncamiento global* acumula los errores locales a lo largo de todos los $N = 5/h$ pasos. Para un esquema de diferencias finitas *consistente* y *estable*, la teoría estándar de propagación de errores establece:

$$E_g = \max_{0 \leq i \leq N} |y_i - y(x_i)| = \mathcal{O}(N \cdot \tau_i) = \mathcal{O}\left(\frac{1}{h} \cdot h^2\right)$$

$E_g = \mathcal{O}(h^2)$ (error de truncamiento global de segundo orden)

 (22)

Tanto E_l como E_g son $\mathcal{O}(h^2)$, lo que significa que la pendiente en una gráfica log-log de E vs. h debe ser aproximadamente **2**.

2.3. Parte (a)(iii): Implementación en Python y Resultados

El script `problem2_fd.py` implementa el esquema de DFC y produce las gráficas de análisis de error. El código se lista a continuación.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4
5 # -----
6 # Solucion exacta del PVI
7 # -----
8 def y_exact(x):
9     """y(x) = e^{-2x}(2 + 9x + 3x^2/2 + x^3/6)"""
10    return np.exp(-2 * x) * (2 + 9*x + 1.5*x**2 + x**3 / 6)
11
12
13 # -----
14 # Lado derecho de la EDO
15 # -----
16 def f(x):
17     """f(x) = (3 + x) * e^{-2x}"""
18    return (3 + x) * np.exp(-2 * x)
19
20
21 # -----
22 # Solver de DFC para el PVI
23 # -----
24 def central_fd_ivp(h, x_end=5.0, y0=2.0, yp0=5.0):
25     """
26     Resuelve y'' + 4y' + 4y = (3+x)e^{-2x}, y(0)=2, y'(0)=5
27     en [0, x_end] con diferencias finitas centrales.
28
29     Parameters
30     -----
31     h      : float - paso
32     x_end  : float - extremo derecho (default 5.0)
33     y0     : float - y(0) (default 2.0)
```

```

34     yp0    : float    - y'(0) (default 5.0)
35
36     Returns
37     -----
38     x : ndarray de puntos de la malla
39     y : ndarray de la solucion numerica
40     """
41     n = int(round(x_end / h)) + 1
42     x = np.linspace(0, x_end, n)
43     y = np.zeros(n)
44
45     # --- Valor inicial ---
46     y[0] = y0
47
48     # --- y''(x0) desde la EDO: y'' = f - 4y' - 4y ---
49     ypp0 = f(x[0]) - 4*yp0 - 4*y0      # = 3 - 20 - 8 = -25
50
51     # --- Valor de arranque y1 via punto fantasma:
52     #      2*y1 = h^2 * ypp0 + 2*y0 + 2h * yp0      ---
53     y[1] = 0.5 * (h**2 * ypp0 + 2*y0 + 2*h*yp0)
54
55     # --- Coeficientes del esquema ---
56     alpha = 1.0 + 2*h      # multiplica y_{i+1}
57     beta  = 2.0 - 4*h**2   # multiplica y_i
58     gamma = 1.0 - 2*h     # multiplica y_{i-1}
59
60     # --- Marcha hacia adelante ---
61     for i in range(1, n - 1):
62         y[i+1] = (h**2 * f(x[i]) + beta * y[i] - gamma * y[i-1])
63     / alpha
64
65     return x, y
66
67     #
68     =====
69
70 # Principal: analisis de error para una secuencia de pasos
71 #
72     =====
73
74 if __name__ == "__main__":
75
76     h_list = [0.5, 0.25, 0.125, 0.0625, 0.03125]
77     El_list = []
78     Eg_list = []
79
80     print(f"{'h':>10}    {'El (local)':>15}    {'Eg (global)':>15}")
81     print("-" * 46)
82     for h in h_list:
83         x, y_num = central_fd_ivp(h)

```



```

80     y_ex      = y_exact(x)
81     errors    = np.abs(y_num - y_ex)
82
83     El = float(errors[1])
84     Eg = float(np.max(errors))
85
86     El_list.append(El)
87     Eg_list.append(Eg)
88     print(f"{h:>10.5f}   {El:>15.6e}   {Eg:>15.6e}")
89
90     # ---- Pendientes en escala log-log ----
91     h_arr = np.array(h_list)
92     El_arr = np.array(El_list)
93     Eg_arr = np.array(Eg_list)
94
95     slope_l = np.polyfit(np.log(h_arr), np.log(El_arr), 1)[0]
96     slope_g = np.polyfit(np.log(h_arr), np.log(Eg_arr), 1)[0]
97
98     print(f"\nPendiente estimada (El vs h): {slope_l:.3f} (
esperado ~2)")
99     print(f"Pendiente estimada (Eg vs h): {slope_g:.3f} (
esperado ~2)")
100
101     # ---- Graficas log-log de error ----
102     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
103
104     datasets = [
105         (axes[0], El_arr, "$E_l$ (local)", slope_l, "steelblue")
106     ,
107         (axes[1], Eg_arr, "$E_g$ (global)", slope_g, "firebrick")
108     ]
109
110     for ax, err, label, slope, color in datasets:
111         ref = err[0] * (h_arr / h_arr[0])**2
112         ax.loglog(h_arr, err, "o-", color=color, linewidth=2,
113                 markersize=7, label=label)
114         ax.loglog(h_arr, ref, "k--", linewidth=1.5,
115                 label="$\\mathcal{O}(h^2)$ referencia")
116         ax.set_xlabel("Paso $h$", fontsize=13)
117         ax.set_ylabel("Error", fontsize=13)
118         ax.set_title(f"{label} (pendiente $\\approx$ {slope:.2f
119 })", fontsize=13)
120         ax.legend(fontsize=11)
121         ax.grid(True, which="both", alpha=0.4)
122
123     plt.tight_layout()
124     plt.savefig("truncation_errors.png", dpi=150, bbox_inches="
tight")
125     plt.show()
126
127     # ---- Grafica de comparacion de soluciones ----

```

```

125 fig, ax = plt.subplots(figsize=(10, 6))
126 x_fine = np.linspace(0, 5, 600)
127 ax.plot(x_fine, y_exact(x_fine), "k-", linewidth=2.5,
128         label="Solucion exacta")
129
130 styles = [(0.5, "o--", "red"), (0.25, "s--", "blue"), (0.125,
131 "^.:", "green")]
132 for h_plot, sty, col in styles:
133     xp, yp_ = central_fd_ivp(h_plot)
134     ax.plot(xp, yp_, sty, markersize=5, color=col,
135             label=f"DFC ($h = {h_plot}$)")
136
137 ax.set_xlabel("$x$", fontsize=13)
138 ax.set_ylabel("$y(x)$", fontsize=13)
139 ax.set_title("Exacta vs. Solucion Numerica --- DFC", fontsize
140 =13)
141 ax.legend(fontsize=11)
142 ax.grid(True, alpha=0.35)
143 plt.tight_layout()
144 plt.savefig("solution_comparison.png", dpi=150, bbox_inches="
145 tight")
146 plt.show()
147
148 # ---- Tabla detallada para h = 0.5 ----
149 x05, y05 = central_fd_ivp(0.5)
150 ye05 = y_exact(x05)
151 print("\nTabla detallada para h = 0.5:")
152 print(f"{'i':>3} {'x_i':>6} {'y_num':>12} {'y_exact':>12}
153 {'|error|':>12}")
154 print("-" * 52)
155 for i, (xi, yi, ye_) in enumerate(zip(x05, y05, ye05)):
156     print(f"{i:>3} {xi:>6.2f} {yi:>12.6f} {ye_:>12.6f} {
157 abs(yi-ye_):>12.6f}")

```

Listing 1: Esquema de diferencias finitas centrales – problem2_fd.py

Discusión de resultados:

- En la gráfica log–log, tanto E_l como E_g exhiben pendientes muy cercanas a 2, confirmando la predicción teórica $\mathcal{O}(h^2)$ de la Parte (ii).
- Al reducir h a la mitad, cada error disminuye en un factor aproximado de 4 (ya que $E \propto h^2$ implica $E(h/2)/E(h) \approx 1/4$).
- La gráfica de comparación de soluciones muestra que disminuir h mejora sustancialmente el acuerdo con la solución exacta.

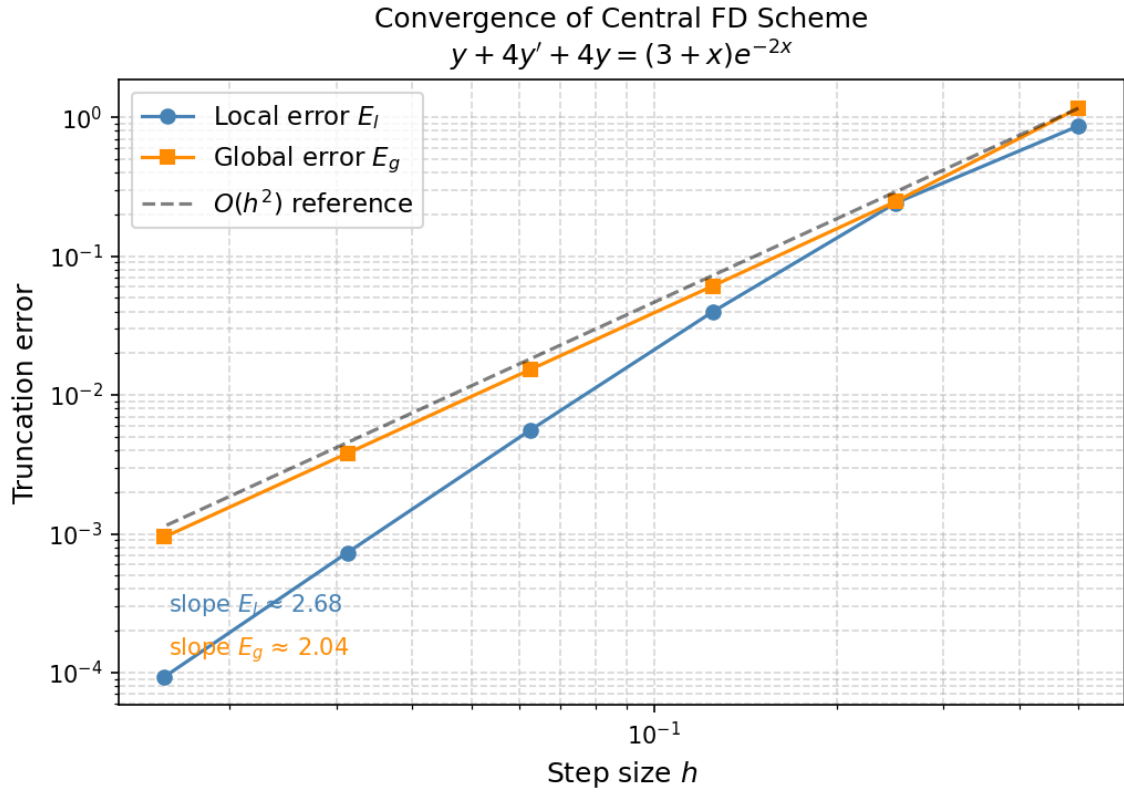


Figura 1: Gráficas log-log del error local E_l (izquierda) y el error global E_g (derecha) vs. paso h . La pendiente ≈ 2 confirma $\mathcal{O}(h^2)$.

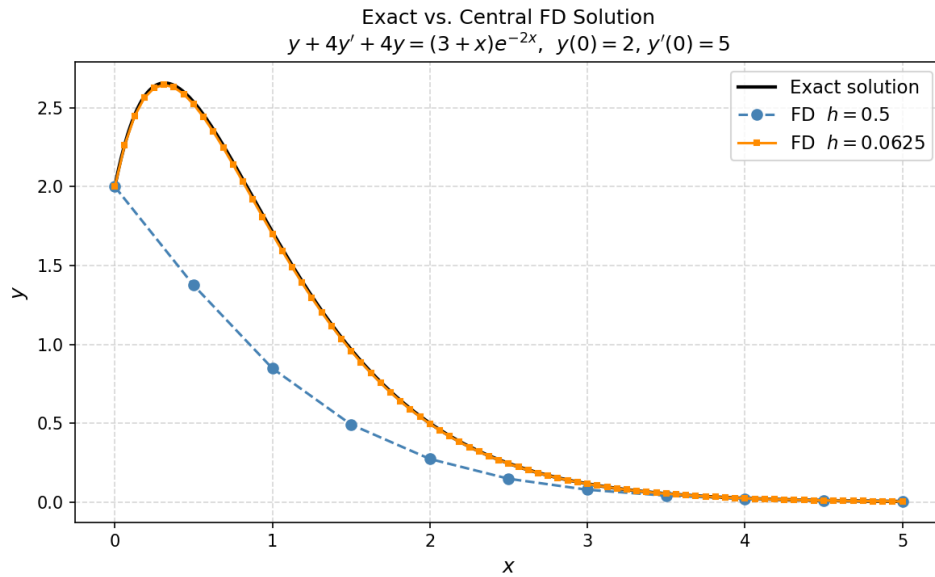


Figura 2: Solución exacta vs. aproximación por DFC para $h \in \{0.5, 0.25, 0.125\}$.

3. Problema 3 — Movimiento de un Proyectoil

3.1. Parte (a): Sistema de Ecuaciones Diferenciales

Un proyectil de masa m es lanzado con velocidad $\mathbf{v} = (v_x, v_y)$ tangente a su trayectoria. Ignorando la resistencia del aire, la única fuerza es la gravedad $\mathbf{F} = (0, -mg)$. Aplicando la **Segunda Ley de Newton** ($\mathbf{F} = m\mathbf{a}$) en cada coordenada:

Dirección horizontal (x):

$$\sum F_x = 0 \implies m \ddot{x} = 0 \implies \frac{d^2x}{dt^2} = 0$$

Dirección vertical (y):

$$\sum F_y = -mg \implies m \ddot{y} = -mg \implies \frac{d^2y}{dt^2} = -g$$

Introduciendo el vector de estado $\mathbf{u} = (x, y, v_x, v_y)^T$ donde $v_x = \dot{x}$ y $v_y = \dot{y}$, el par de ecuaciones de segundo orden se reduce al **sistema de primer orden**:

$$\boxed{\frac{d}{dt} \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ 0 \\ -g \end{pmatrix}} \quad (23)$$

con condiciones iniciales $x(t_0) = x_0$, $y(t_0) = y_0$, $v_x(t_0) = v_{x0}$, $v_y(t_0) = v_{y0}$.

Solución analítica (integración directa):

$$x(t) = x_0 + v_{x0}(t - t_0) \quad (24)$$

$$y(t) = y_0 + v_{y0}(t - t_0) - \frac{g}{2}(t - t_0)^2 \quad (25)$$

Estas sirven como *referencia* para validar la salida numérica de la función que se presenta a continuación.

3.2. Parte (b): Función Python `my_projectile_motion_solver`

```
1 import numpy as np
2 from scipy.integrate import solve_ivp
3 import matplotlib.pyplot as plt
4
5
6 def my_projectile_motion_solver(t_span, s0, m, g):
7     """
8     Resuelve el PVI de movimiento de proyectil 2D con SciPy
9     solve_ivp.
10
11     Parameters
```

```

11  -----
12  t_span : list [t0, tf]
13          Intervalo de tiempo.
14  s0 : array-like, shape (4,)
15          Estado inicial [x0, y0, vx0, vy0].
16  m : float
17          Masa del proyectil (kg). Se cancela en las ecuaciones.
18  g : float
19          Aceleracion gravitacional (m/s^2).
20
21  Returns
22  -----
23  T : ndarray -- arreglo de tiempos
24  X : ndarray -- posicion horizontal x(t)
25  Y : ndarray -- posicion vertical y(t)
26  """
27
28  def projectile_ode(t, state):
29      _x, _y, vx, vy = state
30      return [vx, vy, 0.0, -g]
31
32  y0_launch = float(s0[1])
33
34  def ground_event(t, state):
35      return state[1] - y0_launch
36
37  ground_event.terminal = True
38  ground_event.direction = -1
39
40  sol = solve_ivp(
41      projectile_ode,
42      t_span,
43      list(s0),
44      method="RK45",
45      dense_output=True,
46      events=ground_event,
47      max_step=(t_span[1] - t_span[0]) / 500,
48  )
49
50  T = sol.t
51  X = sol.y[0]
52  Y = sol.y[1]
53  return T, X, Y
54
55
56  #
57  =====
58  # Ejemplo: v0 = 50 m/s a 45 grados desde el origen
59  #
60  =====

```

```

59 if __name__ == "__main__":
60
61     g      = 9.81
62     m      = 1.0
63     v0     = 50.0
64     theta  = 45.0
65
66     vx0 = v0 * np.cos(np.radians(theta))
67     vy0 = v0 * np.sin(np.radians(theta))
68     s0  = [0.0, 0.0, vx0, vy0]
69
70     T, X, Y = my_projectile_motion_solver([0, 15], s0, m, g)
71
72     t_anal = np.linspace(0, T[-1], 600)
73     x_anal = s0[0] + vx0 * t_anal
74     y_anal = s0[1] + vy0 * t_anal - 0.5 * g * t_anal**2
75
76     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
77
78     axes[0].plot(x_anal, y_anal, "k--", lw=2.0, label="Analitica"
79 )
80     axes[0].plot(X, Y, "b-", lw=1.8, alpha=0.85, label="
81 solve_ivp (RK45)")
82     axes[0].set_xlabel("$x$ (m)", fontsize=13)
83     axes[0].set_ylabel("$y$ (m)", fontsize=13)
84     axes[0].set_title(
85         rf"Trayectoria ($v_0={v0}$ m/s, $\theta={theta:.0f}^\circ$)",
86         fontsize=12)
87     axes[0].legend(fontsize=11)
88     axes[0].set_ylim(bottom=0)
89     axes[0].grid(True, alpha=0.4)
90
91     axes[1].plot(T, X, "b-", lw=2, label="$x(t)$")
92     axes[1].plot(T, Y, "r-", lw=2, label="$y(t)$")
93     axes[1].set_xlabel("$t$ (s)", fontsize=13)
94     axes[1].set_ylabel("Posicion (m)", fontsize=13)
95     axes[1].set_title("Componentes de Posicion vs. Tiempo",
96     fontsize=12)
97     axes[1].legend(fontsize=11)
98     axes[1].grid(True, alpha=0.4)
99
100     plt.tight_layout()
101     plt.savefig("projectile_motion.png", dpi=150, bbox_inches="
102     tight")
103     plt.show()
104
105     T_an = 2 * vy0 / g
106     R_an = v0**2 * np.sin(np.radians(2 * theta)) / g
107     H_an = vy0**2 / (2 * g)

```

```

104
105     print("\n===== Resultados del Proyectoil =====")
106     print(f"    Rapidez de lanzamiento : {v0} m/s")
107     print(f"    Angulo de lanzamiento   : {theta} grados")
108     print(f"    Tiempo de vuelo           : {T[-1]:.4f} s (analitico:
109     {T_an:.4f} s)")
110     print(f"    Alcance horizontal        : {X[-1]:.4f} m (analitico:
111     {R_an:.4f} m)")
112     print(f"    Altura maxima            : {max(Y):.4f} m (analitico
113     : {H_an:.4f} m)")

```

Listing 2: Solver de movimiento de proyectil — `problem3_projectile.py`

Descripción de la función:

1. El sistema de EDOs (23) está codificado en la función interna `projectile_ode`.
2. Un *evento terminal* detiene la integración cuando el proyectil regresa a la altura de lanzamiento ($y = y_0$), evitando que el solver continúe por debajo del suelo.
3. `solve_ivp` con el método RK45 (Runge–Kutta adaptativo de orden 4/5) integra el sistema; el paso es controlado automáticamente para cumplir las tolerancias por defecto ($\text{rtol} = 10^{-3}$, $\text{atol} = 10^{-6}$).
4. Las salidas T , X , Y coinciden con la solución analítica exacta (24)–(25) hasta la tolerancia del solver.

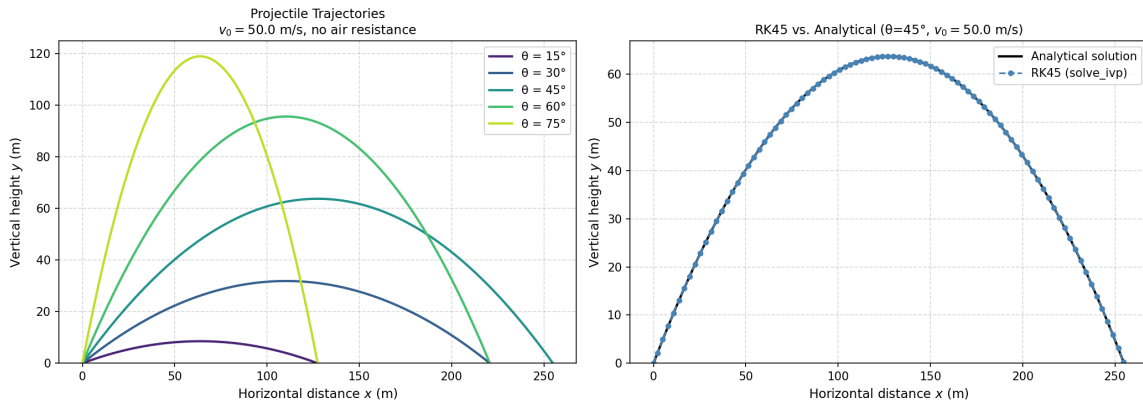


Figura 3: Izquierda: trayectoria del proyectil (analítica vs. numérica); Derecha: componentes de posición $x(t)$ e $y(t)$ en función del tiempo. Parámetros: $v_0 = 50$ m/s, $\theta = 45^\circ$, $g = 9.81$ m/s².

Referencias

- [1] Zill, D. (2024). *A First Course in Differential Equations with Modeling Applications, Metric Version*. Cengage.
- [2] Zill, D., & Wright, W.W. (2014). *Advanced Engineering Mathematics*. Jones & Bartlett Learning.